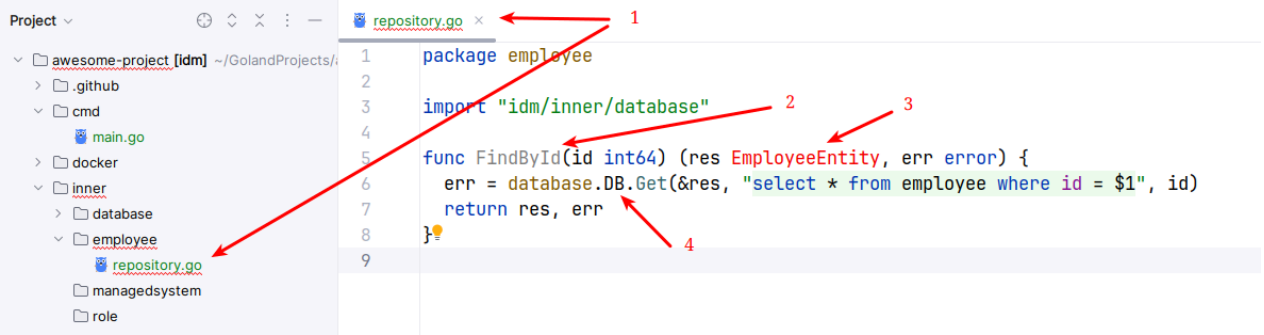
Лекция 12. Repository

Repository - это паттерн. Суть его заключается в том, что поверх хранилища данных создаётся декоратор (репозиторий), который выглядит для всех компонентов приложения как коллекция данных и предоставлет другим компонентам приложения API для работы с этой коллекцией данных.
В микросервисах с трёхслойной архитектурой репозиторий располагается на Data слое. Основное его назначение - скрывать от остального кода детали работы с базой данных (подключение, формирование запросов, маппинг полученных из базы данных строк в структуры и т.д.) и предоставление удобного API для работы с коллекцией данных.
Несмотря на то, что база данных у нас общая для всех доменных сущностей, правильным считается делать отдельные репозитории для каждой сущности:

- EmployeeRepository
- RoleRepository
- ManagedSystemRepository
  С чем связано такое требование? С удобством и эффективностью. На самом деле, намного удобнее и эффективнее в приложении будет работать со слайсом, содержащим данные конкретного типа:
  var employees = []EmployeeEntity{}
  var roles = []RoleEntity{}
  var managedSystems = []ManagedSystemEntity{}
  чем со слайсом, содержащим данные любого типа:
  var allEntities = []any

Какие есть способы реализовать репозиторий?

- в виде go файла (1) с функциями (2) для работы с коллекцией сущностей (3), взаимодействующими с переменной DB (4), которая лежит у нас в пакете database.



преимущество данного варианта - простота реализации. Недостатком данного варианта является сложность написания тестов, в которых слой репозитория должен быть заменён моками или стабами.

- в виде структуры. Это объекто-ориентированный стиль создания репозитория.

Репозиторий создаётся в виде структуры (1), инкапсулирующей подключение к базе данных (2). Для создания экземпляра структуры репозитория, создаётся функция-конструктор (3). API для работы с коллекцией сущностей реализуется в виде методов структуры репозитория (4).

В остальном, этот вариант реализует такой же способ взаимодействия с базой данных, как и предыдущий.

Какой вариант реализации репозитория вы выберете в вашем рабочем проекте - будете решать вы и ваша команда. Небольшие проекты вполне могут обойтись и репозиторием в виде файла с экспортируемыми функциями. В проектах побольше и посложнее вам может потребоваться реализовать репозиторий в виде структуры. при выборе варианта реализации исходите из проблем, которые решает тот или иной вариант:

- простота и скорость разработки,
- возможность использования моков и стабов вместо реальных репозиториев в тестах.
  На нашем курсе мы пойдём по более сложному варианту реализации репозитория.

Создадим entity для репозиториев.
На скриншотах вы могли заметить, что ИДЕ выделила красным тип EmployeeEntity. Это потому, что мы его ещё не создали. Согласно документации на пакет sqlx (https://github.com/jmoiron/sqlx), мы можем объявить структуру EmployeeEntity следующим образом:

repository.go

```go
package employee

import (
  "github.com/jmoiron/sqlx"
  "time"
)

type EmployeeRepository struct {
  db *sqlx.DB
}

func NewEmployeeRepository(database *sqlx.DB) *EmployeeRepository {
  return &EmployeeRepository{db: database}
}

type EmployeeEntity struct {
  Id        int64     `db:"id"`
  Name      string    `db:"name"`
  CreatedAt time.Time `db:"created_at"`
  UpdatedAt time.Time `db:"updated_at"`
}

func (r *EmployeeRepository) FindById(id int64) (employee EmployeeEntity, err error) {
  err = r.db.Get(&employee, "select * from employee where id = $1", id)
  return employee, err
}

```

В Go принято объявлять структуры в месте их использования. Поэтому мы объявим структуру EmployeeEntity в том же файле, где объявили структуру репозитория EmployeeRepository.
Как вы заметите в следующих уроках, EmployeeEntity будет активно использоваться и в Service-слое с бизнес-логикой. Но объявлять её там будет нарушением требований трёхслойной архитектуры: нижележащий слой (EmployeeRepository) не должен ничего знать о вышестоящем слое (EmployeeService). А ему придётся о нём знать, если мы объявим EmployeeEntity в том

же go файле, что и EmployeeService.
В то же время, EmployeeService вправе знать всё об EmployeeRepository.
Таким образом, архитектурно правильным будет объявить структуру EmployeeEntity в том же go файле, где объявлен EmployeeRepository или в отдельном файле.

Задачи на сегодня:

1. Создать для всех ваших доменных пакетов репозитории в ООП-стиле.
2. В репозиториях реализовать API для работы с коллекциями:
   - добавить новый элемент в коллекцию
   - найти элемент коллекции по его id (этот метод мы реализовали на уроке)
   - найти все элементы коллекции
   - найти слайс элементов коллекции по слайсу их id
   - удалить элемент коллекции по его id
   - удалить слайс элементов по слайсу их id
3. Написать интеграционные тесты, проверяющие работу API всех созданых репзиториев.